AWS SERIES · LAB · IAM + LAMBDA + S3 + DYNAMODB

Lab: A Lambda That Loads a File from S3 into DynamoDB

Create an IAM policy and a role, attach the role to a Lambda function, place a JSON file in S3, run the function, and see every employee saved in DynamoDB — with console, Mac and Windows steps.

PA

Tutor: Prudhvi Akella
Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Services: AWS Lambda · Amazon S3 · Amazon DynamoDB · IAM · CloudWatch Logs
Before this lab: read **AWS IAM for Beginners**. Region: `us-east-1`. Files: `lab_lambda_s3_dynamodb.zip`

1. What you will build
2. Policy → role → Lambda
3. What happens when you press Test
4. The code, step by step
5. The data file
6. The two policy files
7. Before you start
8. Step 0 · Open the lab folder and set two variables
9. Step 1 · Create the DynamoDB table
10. Step 2 · Create the S3 bucket and place the file
11. Step 3 · Create the IAM policy (what the Lambda may do)
12. Step 4 · Create the IAM role and attach the policy
13. Step 5 · Create the Lambda function with the role
14. Step 6 · Run the Lambda
15. Step 7 · Check DynamoDB and the logs
16. Step 8 · Place a second file and run again
17. Break it on purpose: see IAM say no
18. Step 9 · Clean up
19. Troubleshooting
20. Key takeaways

1 What you will build

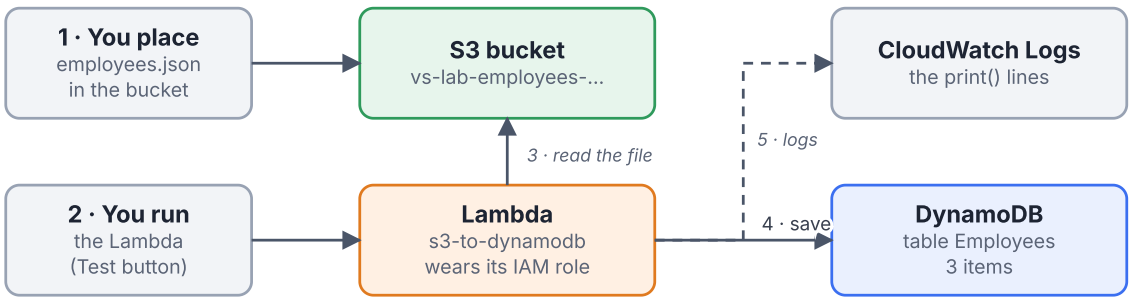


Figure 1. You place a file in S3 and run the Lambda. Wearing its role, it reads the file and saves each employee.

Thing	Name
DynamoDB table	`Employees` — key `emp_id` (String)

Thing	Name
S3 bucket	`vs-lab-employees-<your account id>`
IAM policy	`s3-to-dynamodb-policy`
IAM role	`s3-to-dynamodb-role`
Lambda function	`s3-to-dynamodb` (Python 3.14)

> This lab is IAM in action: the Lambda can read the file and write the table **only** because of the policy you write and the role you attach.

2 Policy → role → Lambda

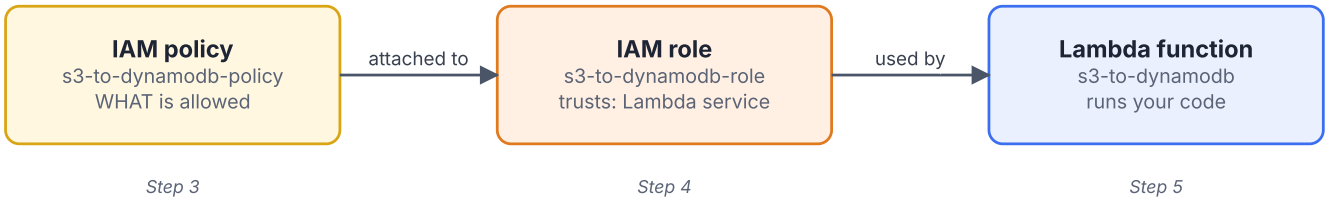


Figure 2. The policy is attached to the role; the role is attached to the Lambda. That chain gives the Lambda its permissions.

1. **Step 3 — the policy:** a JSON list of what is allowed. On its own it does nothing.
2. **Step 4 — the role:** you attach the policy to a role that trusts the Lambda service.
3. **Step 5 — the Lambda:** you pick that role as the function's **execution role**. Now the code has the policy's permissions.

3 What happens when you press Test

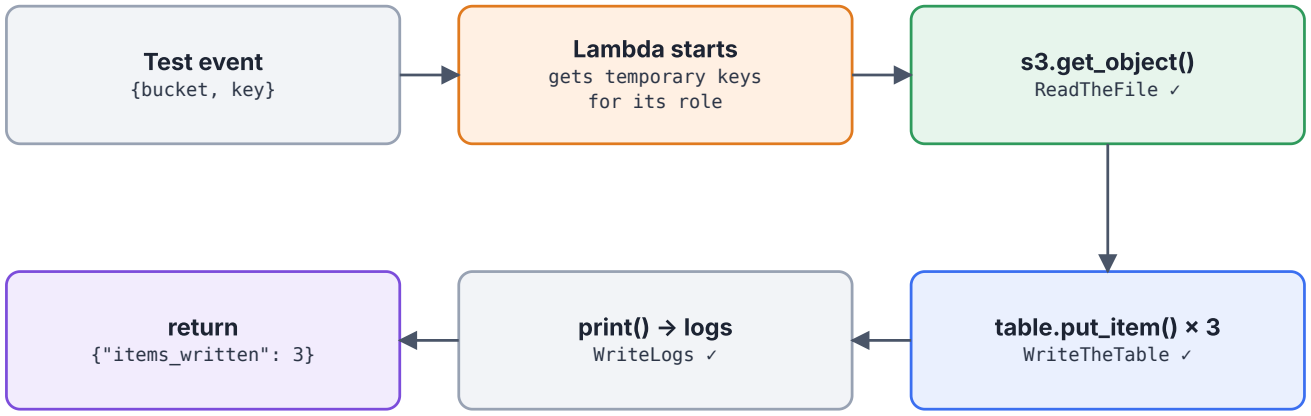


Figure 3. One run, step by step. Every AWS call is checked against the role's policy.

1. **The test event** tells the function the bucket and the file name.

2. **Lambda starts the code** and gives it temporary keys for `s3-to-dynamodb-role`. No key is written anywhere.
3. **Reading the file** is checked by IAM: is `s3:GetObject` allowed on this file? The `ReadTheFile` statement says yes.
4. **Saving each employee** is checked the same way: `dynamodb:PutItem` on `Employees` — `WriteTheTable` says yes.
5. **Log lines** from `print()` go to CloudWatch Logs — allowed by `WriteLogs`.
6. **The return value** is shown in the console under the green result box.

4 The code, step by step

The whole function is `lambda_function.py` in the lab folder. Read the comment at the top of the file first — it shows the same four steps. The code:

```
import json
import os
from decimal import Decimal

import boto3

# Created once when the function starts, then reused every time it runs.
TABLE_NAME = os.environ["TABLE_NAME"]
s3 = boto3.client("s3")
table = boto3.resource("dynamodb").Table(TABLE_NAME)


def lambda_handler(event, context):
    # STEP 1 — which file?
    bucket = event["bucket"]
    key = event["key"]
    print(f"Reading s3://{bucket}/{key}")

    # STEP 2 — download the file content from S3
    response = s3.get_object(Bucket=bucket, Key=key)
    text = response["Body"].read().decode("utf-8")

    # STEP 3 — JSON text → list of dicts.
    # DynamoDB does not accept Python floats, so numbers like 4.5 are read as Decimal.
    employees = json.loads(text, parse_float=Decimal)

    # STEP 4 — save each employee as one item
    for employee in employees:
        table.put_item(Item=employee)
        print(f"Saved {employee['emp_id']} {employee['name']}")

    print(f"{len(employees)} employees written to {TABLE_NAME}")
    return {"file": key, "items_written": len(employees)}
```

Step	Code	Needs (IAM)
1	`event["bucket"]`, `event["key"]` — which file	—
2	`s3.get_object(Bucket=…, Key=…)` — download it	`s3:GetObject`
3	`json.loads(text, parse_float=Decimal)` — text → list of dicts	—

Step	Code	Needs (IAM)
4	`table.put_item(Item=employee)` — save one employee	`dynamodb:PutItem`

Line	Why it is there
`TABLE_NAME = os.environ["TABLE_NAME"]`	The table name comes from a setting (Step 5), so the code works for any table
`s3 = boto3.client("s3")` outside the function	Created once when the Lambda starts, then reused for every run
`parse_float=Decimal`	DynamoDB does not accept Python floats. Without it, `"rating": 4.5` fails with `TypeError: Float types are not supported. Use Decimal types instead.`
`def lambda_handler(event, context)`	Lambda calls this function. `event` is your test event; `context` has run details (unused here)

5 The data file

`data/employees.json` is a JSON list. Each object becomes one DynamoDB item. Every object **must** have `emp_id`, the table's key.

```
[
  {"emp_id": "E1001", "name": "Anita", "department": "HR",      "city": "Hyderabad",
"leave_balance": 18, "rating": 4.5},
  {"emp_id": "E1002", "name": "Ravi",  "department": "Finance", "city": "Mumbai",
"leave_balance": 12, "rating": 4.1},
  {"emp_id": "E1003", "name": "Sana",  "department": "IT",      "city": "Pune",
"leave_balance": 20, "rating": 4.8}
]
```

emp_id (key)	name	department	city	leave_balance	rating
E1001	Anita	HR	Hyderabad	18	4.5
E1002	Ravi	Finance	Mumbai	12	4.1
E1003	Sana	IT	Pune	20	4.8

In DynamoDB, text is stored as type `S` (string) and numbers as type `N`. `data/new-hires.json` adds E1004 Kiran and E1005 Meera in Step 8.

6 The two policy files

policies/permissions-policy.json — what the role may do (Step 3)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadTheFile",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::BUCKET_NAME/*"
    },
    {
      "Sid": "WriteTheTable",
      "Effect": "Allow",
      "Action": "dynamodb:PutItem",
      "Resource": "arn:aws:dynamodb:us-east-1:ACCOUNT_ID:table/Employees"
    },
    {
      "Sid": "WriteLogs",
      "Effect": "Allow",
      "Action": ["logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"],
      "Resource": "*"
    }
  ]
}
```

Before using it, replace `BUCKET_NAME` with your bucket (e.g. `vs-lab-employees-123456789012`) and `ACCOUNT_ID` with your 12-digit account ID. The command-line steps do this for you.

policies/trust-policy.json — who may use the role (Step 4)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "lambda.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

In the console you never paste the trust policy: choosing **AWS service → Lambda** when creating the role writes exactly this for you.

7 Before you start

- An AWS account, signed in as an IAM user with admin rights (not root).
- Unzip `lab_lambda_s3_dynamodb.zip` into your **Downloads** folder.
- Do each step **either** in the console **or** with the commands — not both. Beginners: use the console.

- Commands only: install **AWS CLI v2** and run `aws configure` once. Open the **PyCharm Terminal** (Mac: zsh, Windows: PowerShell) and keep the **same window** open for all steps — later steps use the variables (`$ACCOUNT` , `$BUCKET` , `$POLICY_ARN` , `$ROLE_ARN`) from earlier ones.

> Cost: a few test runs of Lambda, S3 and DynamoDB cost a fraction of a rupee, and the free tier covers it. Still do Step 9 (clean up) at the end.

8 Step 0 · Open the lab folder and set two variables

Every later command uses your account ID and bucket name. We store them once.

Console

1. Sign in to the AWS console.
2. Top right, set the Region to **US East (N. Virginia) us-east-1**.
3. Click your account name (top right) and copy the 12-digit **Account ID** — you need it for the bucket name and the policy.

Command line — Mac

```
cd ~/Downloads/lab_lambda_s3_dynamodb
aws configure set region us-east-1

ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
BUCKET="vs-lab-employees-$ACCOUNT"
echo $ACCOUNT $BUCKET
```

Command line — Windows (PowerShell)

```
cd ~\Downloads\lab_lambda_s3_dynamodb
aws configure set region us-east-1

$ACCOUNT = aws sts get-caller-identity --query Account --output text
$BUCKET = "vs-lab-employees-$ACCOUNT"
echo $ACCOUNT $BUCKET
```

> **You should see:** Two values printed, e.g. `123456789012 vs-lab-employees-123456789012` .

9 Step 1 · Create the DynamoDB table

This is where the employees will be saved.

Console

1. Open **DynamoDB** → **Tables** → **Create table**.
2. Table name: `Employees` . Partition key: `emp_id` , type **String**.
3. Leave **Default settings** → **Create table**. Wait until the status is **Active**.

Command line — Mac

```
aws dynamodb create-table --table-name Employees --attribute-definitions
"AttributeName=emp_id,AttributeType=S" --key-schema "AttributeName=emp_id,KeyType=HASH" --billing-
mode PAY_PER_REQUEST
aws dynamodb wait table-exists --table-name Employees
```

Command line — Windows (PowerShell)

```
aws dynamodb create-table --table-name Employees --attribute-definitions
"AttributeName=emp_id,AttributeType=S" --key-schema "AttributeName=emp_id,KeyType=HASH" --billing-
mode PAY_PER_REQUEST
aws dynamodb wait table-exists --table-name Employees
```

You should see: Table `Employees` is **Active** and has 0 items.

10 Step 2 · Create the S3 bucket and place the file

The Lambda will read `employees.json` from this bucket.

Console

1. Open **S3** → **Create bucket**.
2. Bucket name: `vs-lab-employees-<your account id>` (names must be unique in the whole world). Keep all other defaults → **Create bucket**.
3. Open the bucket → **Upload** → **Add files** → pick `data/employees.json` from the lab folder → **Upload**.

Command line — Mac

```
aws s3 mb "s3://$BUCKET"
aws s3 cp data/employees.json "s3://$BUCKET/employees.json"
aws s3 ls "s3://$BUCKET/"
```

Command line — Windows (PowerShell)

```
aws s3 mb "s3://$BUCKET"
aws s3 cp data\employees.json "s3://$BUCKET/employees.json"
aws s3 ls "s3://$BUCKET/"
```

You should see: The bucket lists one file: `employees.json` .

11 Step 3 · Create the IAM policy (what the Lambda may do)

A policy lists exactly three permissions: read files in this bucket, write to this table, write logs.

Console

1. Open `policies/permissions-policy.json` from the lab folder in a text editor. Replace `BUCKET_NAME` with your bucket name and `ACCOUNT_ID` with your account ID.

2. Open **IAM** → **Policies** → **Create policy** → switch to the **JSON** tab → delete what is there and paste your edited policy → **Next**.
3. Policy name: `s3-to-dynamodb-policy` → **Create policy**.

Command line — Mac

```
sed -e "s/BUCKET_NAME/$BUCKET/" -e "s/ACCOUNT_ID/$ACCOUNT/" policies/permissions-policy.json > my-policy.json
cat my-policy.json

POLICY_ARN=$(aws iam create-policy --policy-name s3-to-dynamodb-policy --policy-document file://my-policy.json --query Policy.Arn --output text)
echo $POLICY_ARN
```

Command line — Windows (PowerShell)

```
(Get-Content policies\permissions-policy.json) -replace "BUCKET_NAME", $BUCKET -replace "ACCOUNT_ID", $ACCOUNT | Set-Content -Encoding ascii my-policy.json
Get-Content my-policy.json

$POLICY_ARN = aws iam create-policy --policy-name s3-to-dynamodb-policy --policy-document file://my-policy.json --query Policy.Arn --output text
echo $POLICY_ARN
```

You should see: `arn:aws:iam::<account id>:policy/s3-to-dynamodb-policy` — the policy exists but nobody uses it yet.

12 Step 4 · Create the IAM role and attach the policy

A role is what the Lambda "wears" when it runs. The policy gives the role its permissions.

Console

1. **IAM** → **Roles** → **Create role**.
2. Trusted entity type: **AWS service**. Use case: **Lambda** → **Next**. (This writes the trust policy for you: "the Lambda service may use this role".)
3. In the search box type `s3-to-dynamodb-policy`, tick it → **Next**.
4. Role name: `s3-to-dynamodb-role` → **Create role**.
5. Open the role and check: **Permissions** tab shows your policy; **Trust relationships** tab shows `lambda.amazonaws.com`.

Command line — Mac

```
ROLE_ARN=$(aws iam create-role --role-name s3-to-dynamodb-role --assume-role-policy-document file://policies/trust-policy.json --query Role.Arn --output text)
aws iam attach-role-policy --role-name s3-to-dynamodb-role --policy-arn $POLICY_ARN

aws iam list-attached-role-policies --role-name s3-to-dynamodb-role
echo $ROLE_ARN
```

Command line — Windows (PowerShell)

```
$ROLE_ARN = aws iam create-role --role-name s3-to-dynamodb-role --assume-role-policy-document
file://policies/trust-policy.json --query Role.Arn --output text
aws iam attach-role-policy --role-name s3-to-dynamodb-role --policy-arn $POLICY_ARN

aws iam list-attached-role-policies --role-name s3-to-dynamodb-role
echo $ROLE_ARN
```

You should see: The role lists one attached policy, `s3-to-dynamodb-policy`.

13 Step 5 · Create the Lambda function with the role

Create the function, give it the role from Step 4, put the code in, and tell it the table name.

Console

1. Open **Lambda** → **Create function** → **Author from scratch**.
2. Function name: `s3-to-dynamodb`. Runtime: **Python 3.14**.
3. Expand **Change default execution role** → **Use an existing role** → pick `s3-to-dynamodb-role` → **Create function**.
4. **Code** tab: open `lambda_function.py`, delete the sample code, paste the code from the lab folder → **Deploy**.
5. **Configuration** → **Environment variables** → **Edit** → **Add environment variable**: key `TABLE_NAME`, value `Employees` → **Save**.
6. **Configuration** → **General configuration** → **Edit** → Timeout `30` sec → **Save**.

Command line — Mac

```
zip function.zip lambda_function.py
sleep 10

aws lambda create-function --function-name s3-to-dynamodb --runtime python3.14 --handler
lambda_function.lambda_handler --role $ROLE_ARN --zip-file fileb://function.zip --timeout 30 --
environment "Variables={TABLE_NAME=Employees}"
aws lambda wait function-active-v2 --function-name s3-to-dynamodb
```

Command line — Windows (PowerShell)

```
Compress-Archive -Path lambda_function.py -DestinationPath function.zip -Force
Start-Sleep -Seconds 10

aws lambda create-function --function-name s3-to-dynamodb --runtime python3.14 --handler
lambda_function.lambda_handler --role $ROLE_ARN --zip-file fileb://function.zip --timeout 30 --
environment "Variables={TABLE_NAME=Employees}"
aws lambda wait function-active-v2 --function-name s3-to-dynamodb
```

You should see: The function exists. **Configuration** → **Permissions** shows the role `s3-to-dynamodb-role`.

`sleep 10` waits for the new role to reach every AWS server. Without it you may see "The role defined for the function cannot be assumed by Lambda" — just run the command again.

14 Step 6 · Run the Lambda

The test event tells the function which file to load.

Console

1. In the function, open the **Test** tab → **Create new event**. Event name: `load-employees`.
2. Replace the event JSON with the block below (use **your** bucket name) → **Save** → **Test**.
3. Open **Details** under the green result box: you see the return value and the log lines.

```
{
  "bucket": "vs-lab-employees-123456789012",
  "key": "employees.json"
}
```

Command line — Mac

```
cat > test-event.json <<EOF
{ "bucket": "$BUCKET", "key": "employees.json" }
EOF

aws lambda invoke --function-name s3-to-dynamodb --cli-binary-format raw-in-base64-out --payload
file://test-event.json response.json
cat response.json
```

Command line — Windows (PowerShell)

```
@"
{ "bucket": "$BUCKET", "key": "employees.json" }
"@ | Set-Content -Encoding ascii test-event.json

aws lambda invoke --function-name s3-to-dynamodb --cli-binary-format raw-in-base64-out --payload
file://test-event.json response.json
Get-Content response.json
```

You should see: `{"file": "employees.json", "items_written": 3}`

15 Step 7 · Check DynamoDB and the logs

See the saved items, and the lines the function printed.

Console

1. **DynamoDB** → **Tables** → `Employees` → **Explore table items**: 3 items, E1001–E1003.
2. **Lambda** → your function → **Monitor** → **View CloudWatch logs** → open the newest log stream.

Command line — Mac

```
aws dynamodb scan --table-name Employees --query "Items[].[emp_id.S, name.S, department.S]" --
output table
aws logs tail /aws/lambda/s3-to-dynamodb --since 10m
```

Command line — Windows (PowerShell)

```
aws dynamodb scan --table-name Employees --query "Items[].[emp_id.S, name.S, department.S]" --
output table
aws logs tail /aws/lambda/s3-to-dynamodb --since 10m
```

You should see: 3 rows (E1001 Anita HR, E1002 Ravi Finance, E1003 Sana IT) and the log lines `Saved E1001 Anita` …

16 Step 8 · Place a second file and run again

Same function, a different file: only the event changes.

Console

1. **S3** → your bucket → **Upload** `data/new-hires.json`.
2. **Lambda** → **Test** tab → change `"key"` to `"new-hires.json"` → **Test**.
3. **DynamoDB** → **Explore table items** → **Run**: now 5 items.

Command line — Mac

```
aws s3 cp data/new-hires.json "s3://$BUCKET/new-hires.json"

cat > test-event.json <<EOF
{ "bucket": "$BUCKET", "key": "new-hires.json" }
EOF
aws lambda invoke --function-name s3-to-dynamodb --cli-binary-format raw-in-base64-out --payload
file://test-event.json response.json
cat response.json

aws dynamodb scan --table-name Employees --select COUNT
```

Command line — Windows (PowerShell)

```
aws s3 cp data\new-hires.json "s3://$BUCKET/new-hires.json"

@"
{ "bucket": "$BUCKET", "key": "new-hires.json" }
"@ | Set-Content -Encoding ascii test-event.json
aws lambda invoke --function-name s3-to-dynamodb --cli-binary-format raw-in-base64-out --payload
file://test-event.json response.json
Get-Content response.json

aws dynamodb scan --table-name Employees --select COUNT
```

You should see: `{"file": "new-hires.json", "items_written": 2}` and `"Count": 5`.

17 Break it on purpose: see IAM say no

Optional, console only, 5 minutes. Remove one permission and watch the Lambda fail — then fix it.

1. **IAM** → **Policies** → `s3-to-dynamodb-policy` → **Edit**.
2. In the JSON, delete the whole `WriteTheTable` block (from its `{` to its `},`) → **Next** → **Save changes**.
3. Wait about 30 seconds, then in **Lambda** → **Test** tab → **Test** again.
4. The result is red. The error says the role is **not authorized to perform: dynamodb:PutItem** on the `Employees` table. The log still shows `Reading s3://…` — reading the file was allowed; saving was not.
5. Go back to the policy → **Edit** → paste the full policy from `my-policy.json` (or your edited copy) → **Save changes**. Wait 30 seconds and **Test** — green again.

> This is the whole idea of IAM: the code did not change at all. Only the policy decided what it could do.

18 Step 9 · Clean up

Delete everything so nothing is left running. Opened a new terminal? Run Step 0 again first.

Console

1. **Lambda** → select `s3-to-dynamodb` → **Actions** → **Delete**.
2. **CloudWatch** → **Log groups** → `/aws/lambda/s3-to-dynamodb` → **Delete**.
3. **IAM** → **Roles** → `s3-to-dynamodb-role` → **Delete**. Then **Policies** → `s3-to-dynamodb-policy` → **Delete**.
4. **S3** → your bucket → **Empty**, then **Delete**.
5. **DynamoDB** → `Employees` → **Delete**.

Command line — Mac

```
aws lambda delete-function --function-name s3-to-dynamodb
aws logs delete-log-group --log-group-name /aws/lambda/s3-to-dynamodb

aws iam detach-role-policy --role-name s3-to-dynamodb-role --policy-arn
"arn:aws:iam::${ACCOUNT}:policy/s3-to-dynamodb-policy"
aws iam delete-role --role-name s3-to-dynamodb-role
aws iam delete-policy --policy-arn "arn:aws:iam::${ACCOUNT}:policy/s3-to-dynamodb-policy"

aws s3 rb "s3://$BUCKET" --force
aws dynamodb delete-table --table-name Employees
```

Command line — Windows (PowerShell)

```
aws lambda delete-function --function-name s3-to-dynamodb
aws logs delete-log-group --log-group-name /aws/lambda/s3-to-dynamodb

aws iam detach-role-policy --role-name s3-to-dynamodb-role --policy-arn
"arn:aws:iam::${ACCOUNT}:policy/s3-to-dynamodb-policy"
aws iam delete-role --role-name s3-to-dynamodb-role
aws iam delete-policy --policy-arn "arn:aws:iam::${ACCOUNT}:policy/s3-to-dynamodb-policy"

aws s3 rb "s3://$BUCKET" --force
aws dynamodb delete-table --table-name Employees
```

You should see: Nothing from this lab is left in your account.

19 Troubleshooting

Error you see	Cause	Fix
`KeyError: 'TABLE_NAME'`	The environment variable is missing	Step 5: add `TABLE_NAME` = `Employees`
`KeyError: 'bucket'`	The test event JSON has a different field name	Use exactly `{"bucket": "…", "key": "…"}`
`NoSuchKey`	The file name in the event is not in the bucket (names are case-sensitive)	Compare the `key` with the file name in S3
`NoSuchBucket`	Bucket name typo	Copy the name from the S3 console
`AccessDenied … s3:GetObject`	The policy's bucket name is wrong, `/*` is missing, or the policy is not attached	Check Step 3 and the role's **Permissions** tab
`AccessDeniedException … dynamodb:PutItem`	Wrong account ID or table name in the policy	Check the ARN in Step 3
`ResourceNotFoundException`	The table doesn't exist in us-east-1, or the name differs	Check the Region (top right) and the table name
`Task timed out after 3.00 seconds`	Timeout was not raised	Step 5: Timeout 30 sec
`The role defined for the function cannot be assumed by Lambda`	The role is brand new, or its trust policy is wrong	Wait 10 seconds and retry; check **Trust relationships**
`Unable to import module 'lambda_function'`	File name or handler doesn't match	File must be `lambda_function.py` with `def lambda_handler`

20 Key takeaways

1. A **policy** lists what is allowed; attach it to a **role**; pick the role as the Lambda's **execution role**.
2. The Lambda never has a password or access keys — AWS gives its role temporary keys on every run.
3. Each AWS call in the code needs one permission: `s3:GetObject` to read, `dynamodb:PutItem` to save, `logs:*` to log.
4. The test event says which file to load; the same function loads any file you name.
5. Remove a permission and the call fails with AccessDenied — without any change to the code.
6. Clean up at the end: function, log group, role, policy, bucket, table.